

Labwork 4: Threads

Pham Tien Nhat

October 1, 2026

1 Implementation

Image is loaded using `matplotlib.pyplot.imread()`.

```
image = plt.imread('1.jpeg')
plt.imshow(image)
h, w, d = image.shape
```

CPU time is extracted to calculate the speedrun.

```
grayCPU = np.zeros((h, w), np.uint8)
start = time.time()
for y in range(h):
    for x in range(w):
        grayCPU[y, x] = np.uint8((int(image[y, x, 0]) + int(image[y, x, 1])
                                + int(image[y, x, 2])) / 3)
cpu_time = time.time() - start
```

In the 2D kernel for GPU implementation, each thread finds their column `tidx` from the x dimension and their row `tidy` from the y dimension.

```
@cuda.jit
def grayscale2D(src, dst):
    tidx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    g = np.uint8((src[tidy, tidx, 0] + src[tidy, tidx, 1]
                  + src[tidy, tidx, 2]) / 3)
    dst[tidy, tidx, 0] = dst[tidy, tidx, 1] = dst[tidy, tidx, 2] = g
```

The block size and the grid size are 2-dimensional tuples.

```
blockSize = (32, 32)
gridSize = (math.ceil(w / blockSize[0]),
            math.ceil(h / blockSize[1]))

devSrc = cuda.to_device(image)
devDst = cuda.device_array((h, w, 3), np.uint8)
grayscale2D[gridSize, blockSize](devSrc, devDst)
hostDst = devDst.copy_to_host()
```

2 Experimental Results

4 2D block sizes are tested: (4,4), (8,8), (16,16) and (32,32), and the speedup is the CPU time divided by the GPU time.

```

block_sizes = [(4, 4), (8, 8), (16, 16), (32, 32)]
speedups = []

for blockSize in block_sizes:
    gridSize = (math.ceil(w / blockSize[0]),
                math.ceil(h / blockSize[1]))
    devDst = cuda.device_array((h, w, 3), dtype=np.uint8)
    start = time.time()
    grayscale2D[gridSize, blockSize](devSrc, devDst)
    cuda.synchronize()
    end = time.time()
    gpu_time = end - start
    speedups.append(cpu_time / gpu_time)

    print(f"Block size: {blockSize}")
    print(f"Grid size: {gridSize}")
    print(f"GPU time: {gpu_time} seconds")

plt.figure(figsize=(8, 5))
plt.plot([str(b) for b in block_sizes], speedups, marker='o')
plt.xlabel("2D Block Size")
plt.ylabel("Speedup")
plt.title("CUDA 2D Block Size vs Speedup")
plt.grid(True)
plt.show()

```

Figure 1 shows the relationship between the 2D block size and the speedup. Speedup is the lowest at (4, 4), and the larger blocks reach about 2250 speedup.

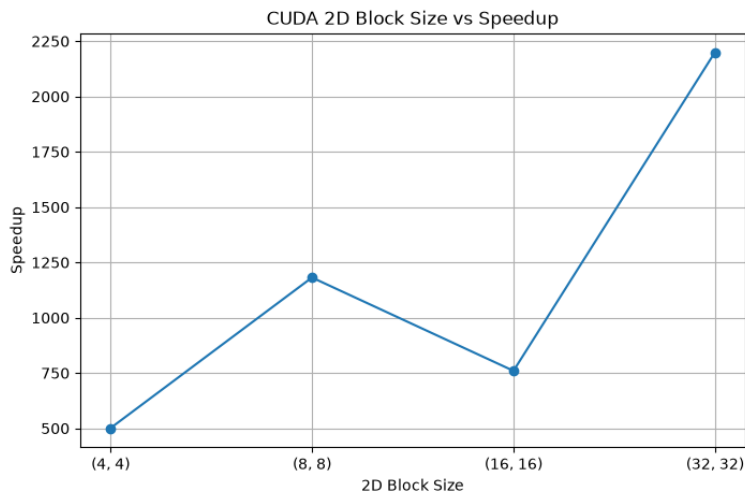


Figure 1: 2D block size vs speedup.

3 Comparison with 1D Grid

Figure 2 is the block size and time graph of Labwork 3, which uses 1D blocks and the same timing method.

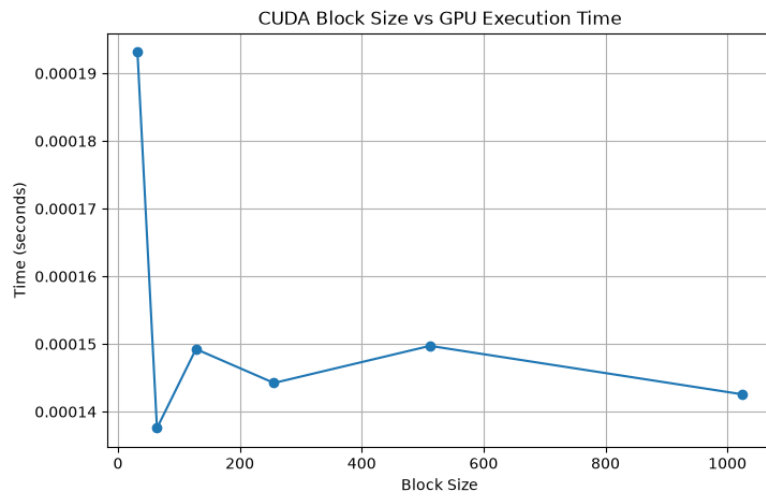


Figure 2: Labwork 3: 1D block size vs kernel time (seconds).